FAST NUCES

Department of Artificial Intelligence

National University of Computer and Emerging Sciences

Islamabad, Pakistan

Operating Systems Project

Submitted By

M-Faisal - 24I-6077
Talha Habib - 24I-0029
Abdul Ghafoor - 24I-0118

May 5, 2026

Problem Statement

Traditional data programs are often slow because they work in a straight line, handling one task at a time. This creates several issues:

- **Wasted Power:** The computer has multiple processor cores, but a single-threaded program only uses one, leaving the rest of the system's power unused.
- **Bottlenecks:** If one file is very large, the entire program has to wait for it to finish before moving to the next task.
- **Memory Issues:** Loading huge datasets at once can crash the program or slow down the entire system.

Solution

We built a **Parallel Data Pipeline on Linux** that breaks the work into smaller tasks and runs them concurrently to improve performance. The system is designed as follows:

- **Dividing the Labor:** The system is split into four separate processes reading, processing, aggregating, and reporting each handling a specific stage of the pipeline.
- **Thread Pool:** Inside the processing stage, a pool of worker threads is used to handle different chunks of data simultaneously, improving execution speed.
- **Smart Communication:** Named Pipes and Shared Memory are used for fast inter-process communication without relying on temporary files.
- **Safe Synchronization:** Mutexes and Semaphores are used to coordinate threads and prevent race conditions while updating shared results.

Tech Stack

- Programming Language: C++
- System Scripting: Shell (Linux Bash)
- Concurrency Libraries: C/POSIX Threading (pthreads)
- IPC Mechanisms: Named Pipes (FIFOs), Shared Memory
- Synchronization Tools: Mutexes, Semaphores
- Operating System: Linux

System Design and Architecture

This system is designed as a parallel data processing pipeline on Linux, where each major component is mapped to separate processes and threads. The architecture is composed of four main modules: data producer (ingester), worker pool (processor), aggregator (table structure), and reporter. Inter-process communication (IPC) and synchronization mechanisms are used to ensure efficient and safe data flow.

1. Shell (Execution Controller)

The shell acts as the execution controller of the system. It is responsible for:

- Ensuring required directories and files are available before execution.
- Configuring runtime parameters such as thread count and queue size.
- Executing the main program and capturing runtime and exit status.
- Handling cleanup using traps, including removal of temporary files and executables.

2. Dispatcher (Process Coordinator)

The dispatcher is the entry point of the system and is responsible for initializing all major components. It:

- Launches and manages all processes in the pipeline.
- Tracks execution time and exit status of each process.
- Creates IPC resources including named pipes, shared memory, and named semaphores.
- Coordinates synchronization between processor and reporter using semaphores to signal data availability.

3. Ingester (Data Producer)

The ingester reads CSV input files and prepares data for processing. Its responsibilities include:

- Reading input CSV files and dividing data into chunks based on thread count.
- Distributing chunks proportionally among available worker threads.
- Sending an EOF (end-of-file) chunk to signal completion of data production.
- Writing data into a named pipe for consumption by the processor.

4. Processor (Worker Pool)

The processor is the core computation unit of the system. It:

- Retrieves data from the named pipe shared by the ingester.
- Uses a pool of worker threads to process data concurrently.
- Synchronizes access to shared queues using mutexes and semaphores.
- Computes aggregated results (e.g., summing revenue per category).
- Stores results in a shared in-memory table structure.

Each worker thread processes chunks of data independently. A **poison pill (NULL chunk)** is used to signal thread termination once all data has been processed.

5. Aggregator (Shared Table Structure)

The aggregated results are stored in a shared table structure:

- Ensures unique category indexing.
- Merges duplicate categories by summing their values.
- Provides a centralized and thread-safe storage mechanism.

6. Reporter

The reporter is responsible for generating final output after processing is complete:

- Waits on a semaphore until data is available in shared memory.
- Reads aggregated data from shared memory.
- Generates output files in CSV and human-readable text format.
- Signals the dispatcher once report generation is complete.

IPC and Synchronization Design

The system uses multiple IPC and synchronization mechanisms:

- **Named Pipes:** Used for streaming data from ingester to processor.
- **Shared Memory:** Used for storing final aggregated results.
- **Named Semaphores:** Used to control access between processor and reporter.
- **Mutexes:** Ensure safe access to shared queues and tables.
- **Poison Pill Mechanism:** Used for graceful termination of worker threads.

Implementation Details

This section explains how the system is configured at runtime, how data is passed between components, and how resources are managed using command-line arguments and controlled execution flow.

1. Runtime Configuration via Shell

The shell acts as the primary interface for configuring the system before execution. By default, the system uses predefined values for input directory, output directory, thread count, and queue size. However, these parameters can be modified dynamically using command-line flags.

- **-D** : Specifies the input directory containing CSV files.
- **-O** : Defines the output directory where results will be stored.
- **-T** : Sets the number of worker threads in the processor stage.
- **-S** : Configures the size of the processing queue.

If no flags are provided, the system automatically falls back to default configuration values defined in the shell script.

2. Data Flow Through the Pipeline

Once the configuration is set, the system executes a structured data flow pipeline:

- The shell initializes execution and passes configuration parameters to the dispatcher.
- The dispatcher forwards these settings to all processes in the pipeline.
- The ingester reads input files from the directory specified by **-D** and begins streaming data.
- Data is passed through a named pipe to the processor stage in chunks.
- The processor distributes data among worker threads based on the configured thread count (**-T**) and queue size (**-S**).
- Processed results are stored in shared memory and later accessed by the reporter.
- The reporter writes final output files to the directory specified by **-O**.

3. Dynamic Resource Binding

All runtime parameters directly affect system behavior:

- Thread pool size is dynamically created based on the **-T** flag.
- Queue capacity is adjusted according to the **-S** flag.
- Input and output paths are resolved at startup and validated before execution.
- If invalid or missing flags are detected, default configuration is applied.

4. Execution Control and Propagation

The dispatcher ensures that all configuration values are consistently propagated across processes:

- A centralized configuration structure is created in the shell.
- This structure is passed to child processes during exec via arguments.
- Then in each process only data retrieved from arguments to use.

5. Resource Finalization

After execution completes:

- The reporter writes final outputs to the specified output directory (**-O**).
- All IPC resources (pipes, shared memory, semaphores) are released.
- The shell performs cleanup using traps to ensure no temporary artifacts remain.

System Diagrams and Visual Representation

```
faisal@DESKTOP-12IDU0F:~/a$ bash run.sh
Child finished: 19995
Child finished: 19996
Shutdown detected, cleaning up...
Ingester process [19995]  Execution time : [0.0119967] Exited with CODE : 0
Processor process [19996]  Execution time : [0.0115743] Exited with CODE : 0
Reporter process [19997]  Execution time : [0.0122584]Exited by signal: 15
Cleanup done
Total runtime: .036890301 seconds
Records processed: 5
Exit status: 0
Bye
rm -f dispatcher
rm -f ingester
rm -f processor
rm -f reporter
rm -f pipe1
```

This diagram shows the Dispatcher in action. It highlights how the dispatcher tracks child processes and manages cleanup of the shell.

```
logs > ingester.log
Ingester pid= [19995] Parent pid= [19994] starting data reading
Ingester pid= [19995] Parent pid= [19994] Build Paths for files
Ingester pid= [19995] Parent pid= [19994] sended data from PIPE
Sended data chunk ID : 1 FileID for Chunkdata/retail_transactions.csv Lines :5
Ingester pid= [19995] Parent pid= [19994] sended data from PIPE
Sended data chunk ID : 2 FileID for Chunkdata/retail_transactions.csv Lines :5
Ingester pid= [19995] Parent pid= [19994] sended data from PIPE
Sended data chunk ID : 3 FileID for Chunkdata/retail_transactions.csv Lines :1
Ingester pid= [19995] Parent pid= [19994] write EOF chunk
```

This image represents the ingester process which sends data forward in the pipeline. It shows how data flows through different stages handled by the ingester.

```
logs > processor.log

Token : 150
Token : 180
Token :

Worker THREAD : 139947224852160 Chunk id : 3  FileID : data/retail_transactions.csv  Lines no : 1  Data : Fu

Aggregated line : Furniture,400,420,450,,

Token : 400
Token : 420
Token : 450
Token :

Worker thread id : 139947224852160 receiveing signal to stop
Worker thread id : 139947216459456 receiveing signal to stop

==================================

===== AGGREGATION TABLE =====
Electronics : 1800
Clothing : 585
Groceries : 255
Furniture : 2350
============================
```

This figure illustrates the worker thread pool inside the processor module. It demonstrates how multiple threads handle chunks of data in parallel and update the aggregated table.

```
logs > reporter.log
Reporter pid= [19997] Parent pid= [19994] waiting for shared memory lock
Reporter pid= [19997] Parent pid= [19994] aquired shared memory lock
In CSV report/Report.csv
Electronics,1800
Clothing,585
Groceries,255
Furniture,2350
In txt report/Report.txt
=================REPORTER===============

===== AGGREGATION TABLE =====
Electronics : 1800
Clothing : 585
Groceries : 255
Furniture : 2350
```

This diagram shows the reporter process in action. It explains how the reporter wakes up and reads data from shared memory.

Work Division

M-Faisal: Shell and Core Pipeline

- Implemented shell script for system execution and configuration handling.
- Added CLI flags (**-D, -O, -T, -S**) for input/output paths, threads, and queue size.
- Managed environment setup, validation, and cleanup using traps.
- Developed processor using pthreads with worker thread pool, queue handling, and aggregation logic.

Talha Habib: Dispatcher and Path generation

- Developed dispatcher for process orchestration and lifecycle management.
- Implemented IPC using named pipes, shared memory, and semaphores.
- Managed ingester to generate generic file paths to open file and make chunks.

Abdul Ghafoor: Data Processing, Reporter, and Report Writing

- Implemented ingester for CSV reading and data chunking.
- Built reporter module to generate final CSV and text outputs from shared memory.
- Contributed to full project report writing, including system design, implementation details, and final documentation compilation.

Challenges Faced

- **Thread Synchronization Issues:** Multiple worker threads accessed shared data simultaneously, causing race conditions. This was solved using mutexes and semaphores for safe synchronization.

- **Limited Pipe Buffer Capacity:** The named pipe buffer had limited memory capacity, causing the ingester process to mix data when large amounts of data were written faster than the processor could consume them. This issue was handled by sending data in controlled chunks and balancing workload across worker threads to maintain smooth data flow.

- **Load Balancing Between Threads:** Uneven chunk distribution caused some threads to finish earlier than others. Data chunking was adjusted dynamically based on thread count to balance workload.

- **Graceful Thread Termination:** Worker threads sometimes waited indefinitely for new data. A NULL chunk mechanism was introduced to safely terminate all threads after processing completion.

- **Resource Cleanup:** IPC resources such as semaphores and shared memory were not always released properly after crashes. Cleanup traps and controlled shutdown mechanisms were added to ensure proper resource deallocation.

- **Report coruption:** When we tried to write data in Report it was showing null values in file which was due to using sizeof(Data) instead of strlen(Data) in writing.